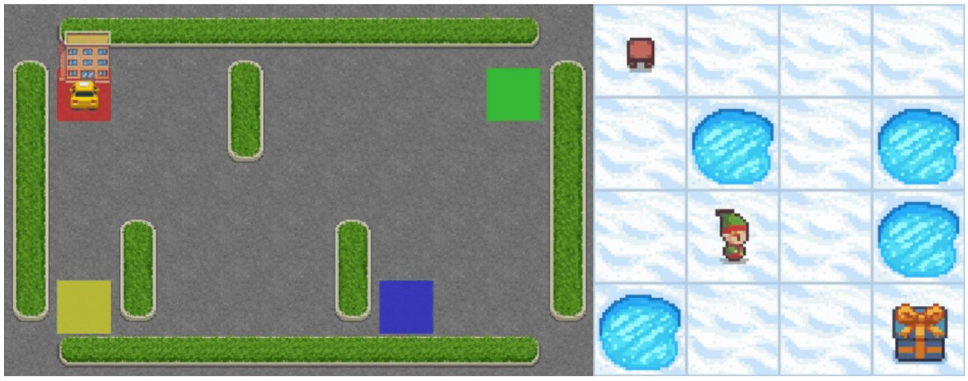
Chapter 2 Experiment Q-Learning

10 May 2024 20:17

Now that we studied the Q-Learning algorithm, let's implement it from scratch and train our Q-Learning agent in two environments:

- Frozen-Lake-v1 (non-slippery and slippery version) : where our agent will need to go from the starting state (S) to the goal state (G) by walking only on frozen tiles (F) and avoiding holes (H).
- An autonomous taxi will need to learn to navigate a city to transport its passengers from point A to point B.



In this section we will use environments: FrozenLake-v1, Taxi-v3. RL-Library: Python and NumPy, Gymnasium

1. Install and import libraries
   a. Install the libraries needed

```
sudo apt-get update
sudo apt-get install -y python3-opengl
apt install ffmpeg xvfb
pip3 install pyvirtualdisplay
```

   b. Import packages

```
import numpy as np
import gymnasium as gym
import random
import imageio
import os
import tqdm

import pickle5 as pickle
from tqdm.notebook import tqdm
```

2. Frozen Lake non slipery
   a. Basic information

   We're going to train our Q-Learning agent **to navigate from the starting state (S) to the goal state (G) by walking only on frozen tiles (F) and avoid holes (H)**.

   We can have two sizes of environment:

   - map_name="4x4": a 4x4 grid version
   - map_name="8x8": a 8x8 grid version

   The environment has two modes:

- is_slippery=False: The agent always moves **in the intended direction** due to the non-slippery nature of the frozen lake (deterministic).
- is_slippery=True: The agent **may not always move in the intended direction** due to the slippery nature of the frozen lake (stochastic).

For now let's keep it simple with the 4x4 map and non-slippery. We add a parameter called render_mode that specifies how the environment should be visualised. In our case because we **want to record a video of the environment at the end, we need to set render_mode to rgb_array**.

b. Create Environment

We see with Observation Space Shape Discrete(16) that the observation is an integer representing the agent's current position as current_row * ncols + current_col (where both the row and col start at 0).

For example, the goal position in the 4x4 map can be calculated as follows: 3 * 4 + 3 = 15. The number of possible observations is dependent on the size of the map. For example, the 4x4 map has 16 possible observations.

```
env = gym.make("FrozenLake-v1", map_name="4x4", is_slippery=False,
render_mode="rgb_array")
```

i. Observation space

```
# We create our environment with gym.make("<name_of_the_environment>")-
`is_slippery=False`: The agent always moves in the intended direction due to the
non-slippery nature of the frozen lake (deterministic).
print("_____OBSERVATION SPACE_____ \n")
print("Observation Space", env.observation_space)
print("Sample observation", env.observation_space.sample())  # Get a random
observation
```

ii. Action Space

```
print("\n _____ACTION SPACE_____ \n")
print("Action Space Shape", env.action_space.n)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

The action space (the set of possible actions the agent can take) is discrete with 4 actions available:

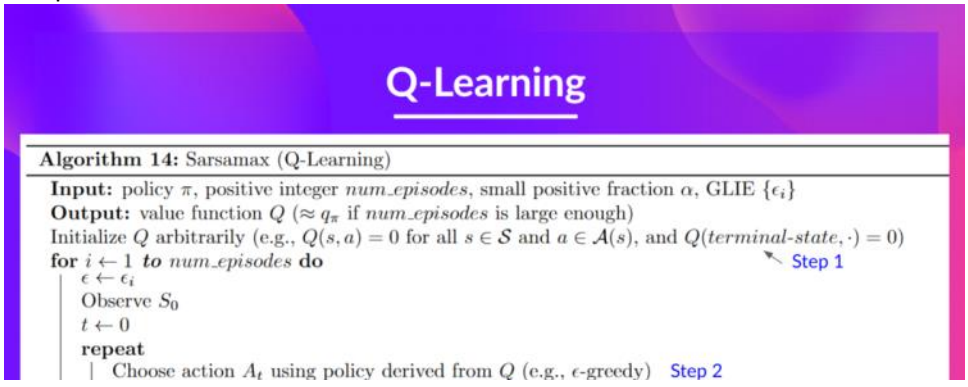
- 0: GO LEFT
- 1: GO DOWN
- 2: GO RIGHT
- 3: GO UP

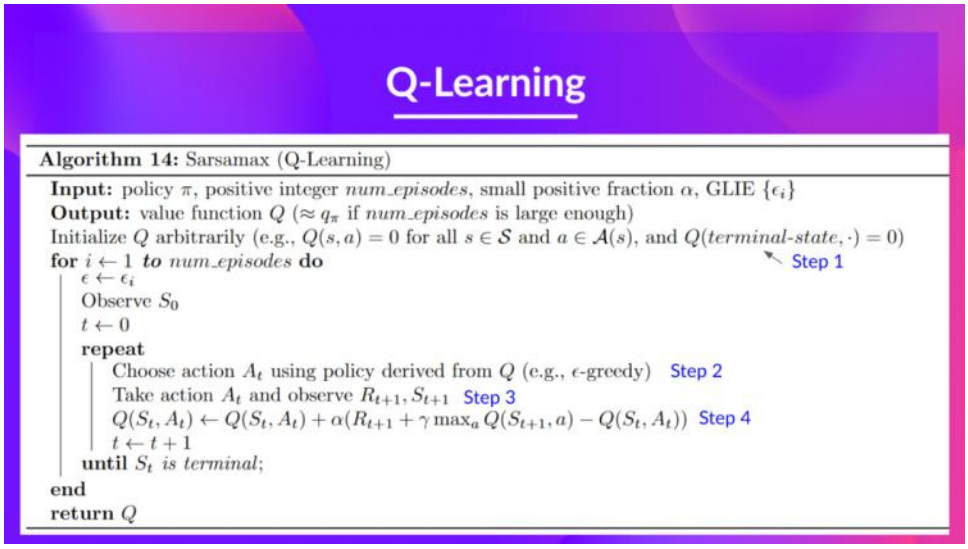
iii. Reward

Reach goal: +1
Reach hole: 0
Reach frozen: 0

c. Create and initialize the Q-Table

i. The pseudo code





ii. The code

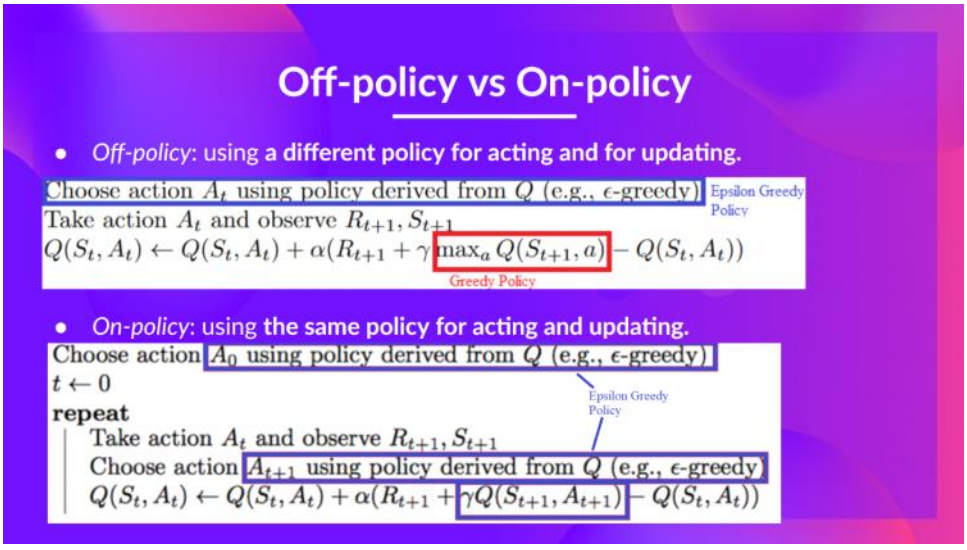
```
state_space = env.observation_space.n
print("There are ", state_space, " possible states")

action_space = env.action_space.n
print("There are ", action_space, " possible actions")

Qtable_frozenlake = initialize_q_table(state_space, action_space)
```

d. Define the Greedy Policy

Remember we have two policies since Q-Learning is an **off-policy** algorithm. This means we're using a **different policy for acting and updating the value function**.

- Epsilon-greedy policy (acting policy)
- Greedy-policy (updating policy)

The greedy policy will also be the final policy we'll have when the Q-learning agent completes training. The greedy policy is used to select an action using the Q-table.



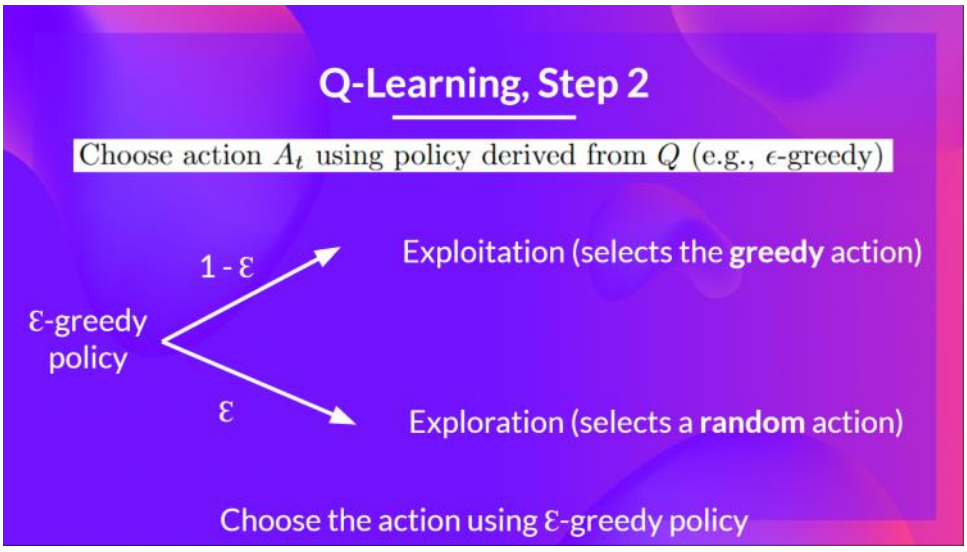

```
def greedy_policy(Qtable, state):
    # Exploitation: take the action with the highest state, action value
    action = np.argmax(Qtable[state][:])

    return action
```

e. Define the Epsilon Greedy Policy

Epsilon-greedy is the training policy that handles the exploration/exploitation trade-off. The idea with epsilon-greedy:

- With *probability 1 - ε* : **we do exploitation** (i.e. our agent selects the action with the highest state-action pair value).
- With *probability ε*: we do **exploration** (trying a random action).

As the training continues, we progressively **reduce the epsilon value since we will need less and less exploration and more exploitation.**



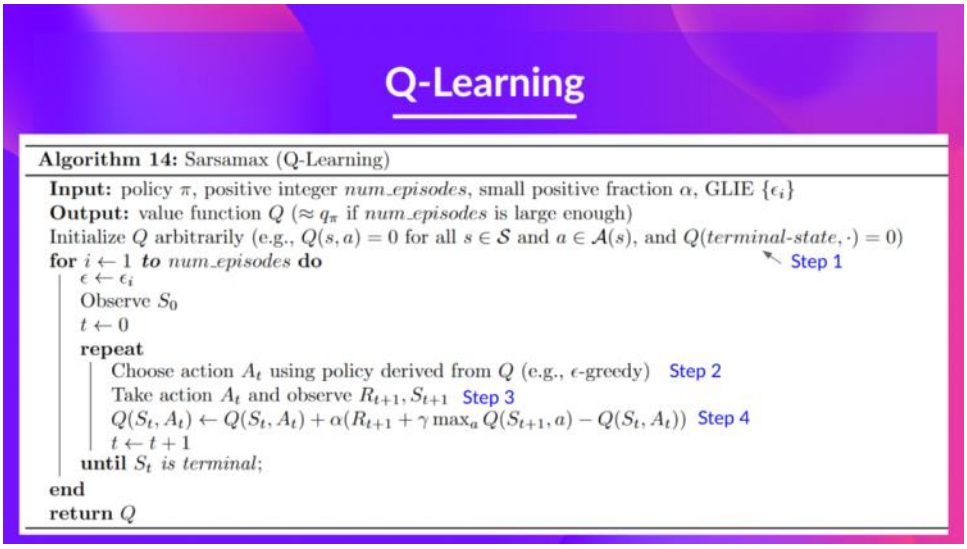

```
def epsilon_greedy_policy(Qtable, state, epsilon):
  # Randomly generate a number between 0 and 1
  random_num = random.uniform(0, 1)
  # if random_num > greater than epsilon --> exploitation
  if random_num > epsilon:
    # Take the action with the highest value given a state
    # np.argmax can be useful here
    action = greedy_policy(Qtable, state)
  # else --> exploration
  else:
    action = env.action_space.sample()

  return action
```

f. Define the Hyperparameter

The exploration related hyperparamters are some of the most important ones.

- We need to make sure that our agent **explores enough of the state space** to learn a good value approximation. To do that, we need to have progressive decay of the epsilon.
- If you decrease epsilon too fast (too high decay_rate), **you take the risk that your agent will be stuck**, since your agent didn't explore enough of the state space and hence can't solve the problem.

g. Create Training Looping Method




```
def train(n_training_episodes, min_epsilon, max_epsilon, decay_rate, env, max_steps,
Qtable):
  for episode in tqdm(range(n_training_episodes)):
    # Reduce epsilon (because we need less and less exploration)
    epsilon = min_epsilon + (max_epsilon - min_epsilon) * np.exp(-decay_rate *
episode)
    # Reset the environment
    state, info = env.reset()
    step = 0
    terminated = False
    truncated = False

    # repeat
    for step in range(max_steps):
      # Choose the action At using epsilon greedy policy
      action = epsilon_greedy_policy(Qtable, state, epsilon)

      # Take action At and observe Rt+1 and St+1
      # Take the action (a) and observe the outcome state(s') and reward (r)
      new_state, reward, terminated, truncated, info = env.step(action)

      # Update Q(s,a):= Q(s,a) + lr [R(s,a) + gamma * max Q(s',a') - Q(s,a)]
      Qtable[state][action] = Qtable[state][action] + learning_rate * (
        reward + gamma * np.max(Qtable[new_state]) - Qtable[state][action]
      )

      # If terminated or truncated finish the episode
      if terminated or truncated:
        break

      # Our next state is the new state
      state = new_state
  return Qtable
```

h. Training the Qlearning Agent

```
Qtable_frozenlake = train(n_training_episodes, min_epsilon, max_epsilon, decay_rate,
env, max_steps, Qtable_frozenlake)
```

i. Evaluate the Q-learning Agent

```
def evaluate_agent(env, max_steps, n_eval_episodes, Q, seed):
```

```
    """
    Evaluate the agent for ``n_eval_episodes`` episodes and returns average reward and
std of reward.
    :param env: The evaluation environment
    :param n_eval_episodes: Number of episode to evaluate the agent
    :param Q: The Q-table
    :param seed: The evaluation seed array (for taxi-v3)
    """
    episode_rewards = []
    for episode in tqdm(range(n_eval_episodes)):
        if seed:
            state, info = env.reset(seed=seed[episode])
        else:
            state, info = env.reset()
        step = 0
        truncated = False
        terminated = False
        total_rewards_ep = 0

        for step in range(max_steps):
            # Take the action (index) that have the maximum expected future reward given
that state
            action = greedy_policy(Q, state)
            new_state, reward, terminated, truncated, info = env.step(action)
            total_rewards_ep += reward

            if terminated or truncated:
                break
            state = new_state
        episode_rewards.append(total_rewards_ep)
    mean_reward = np.mean(episode_rewards)
    std_reward = np.std(episode_rewards)

    return mean_reward, std_reward
```

```
# Evaluate our Agent
mean_reward, std_reward = evaluate_agent(env, max_steps, n_eval_episodes,
Qtable_frozenlake, eval_seed)
print(f"Mean_reward={mean_reward:.2f} +/- {std_reward:.2f}")
```